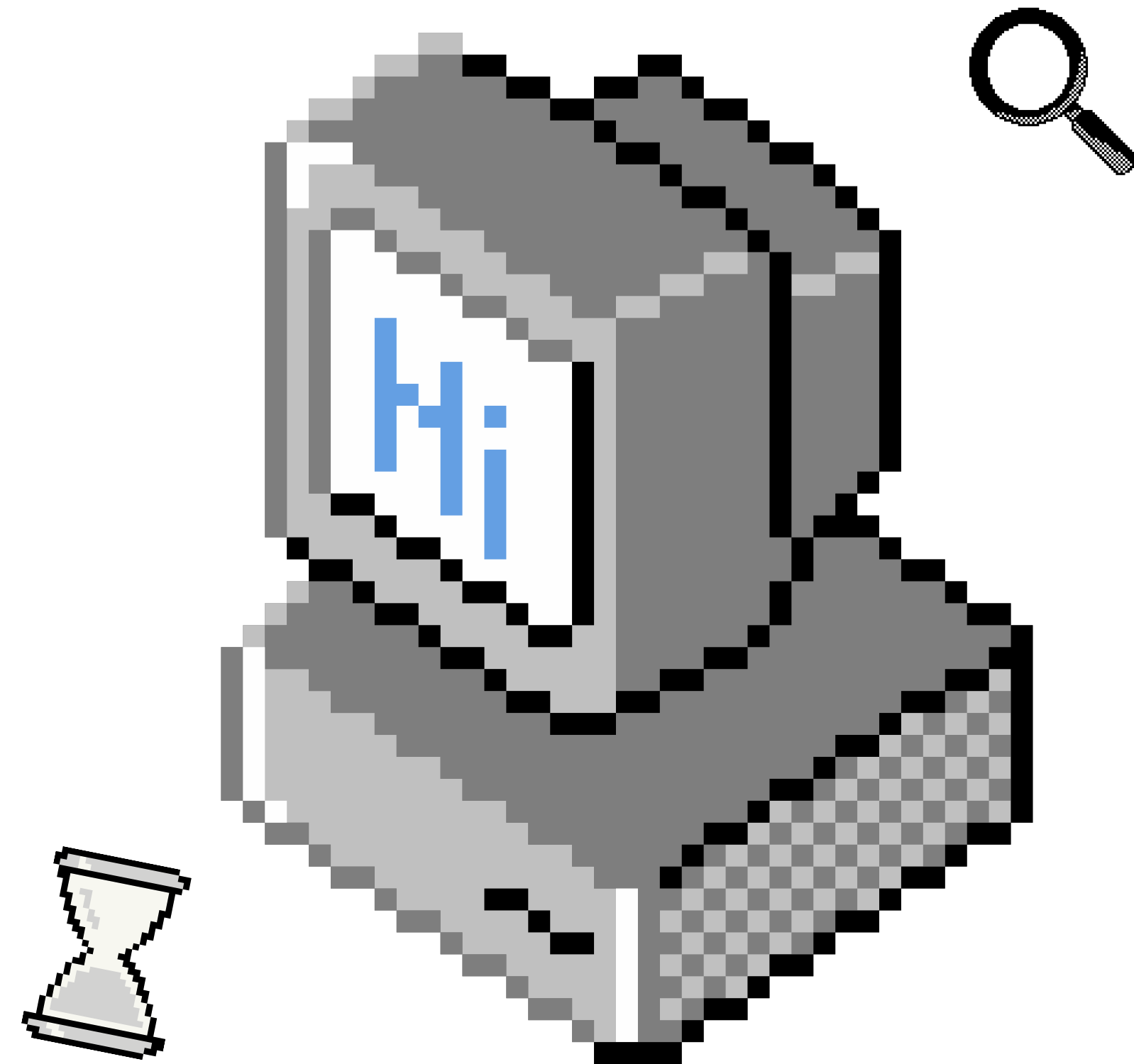




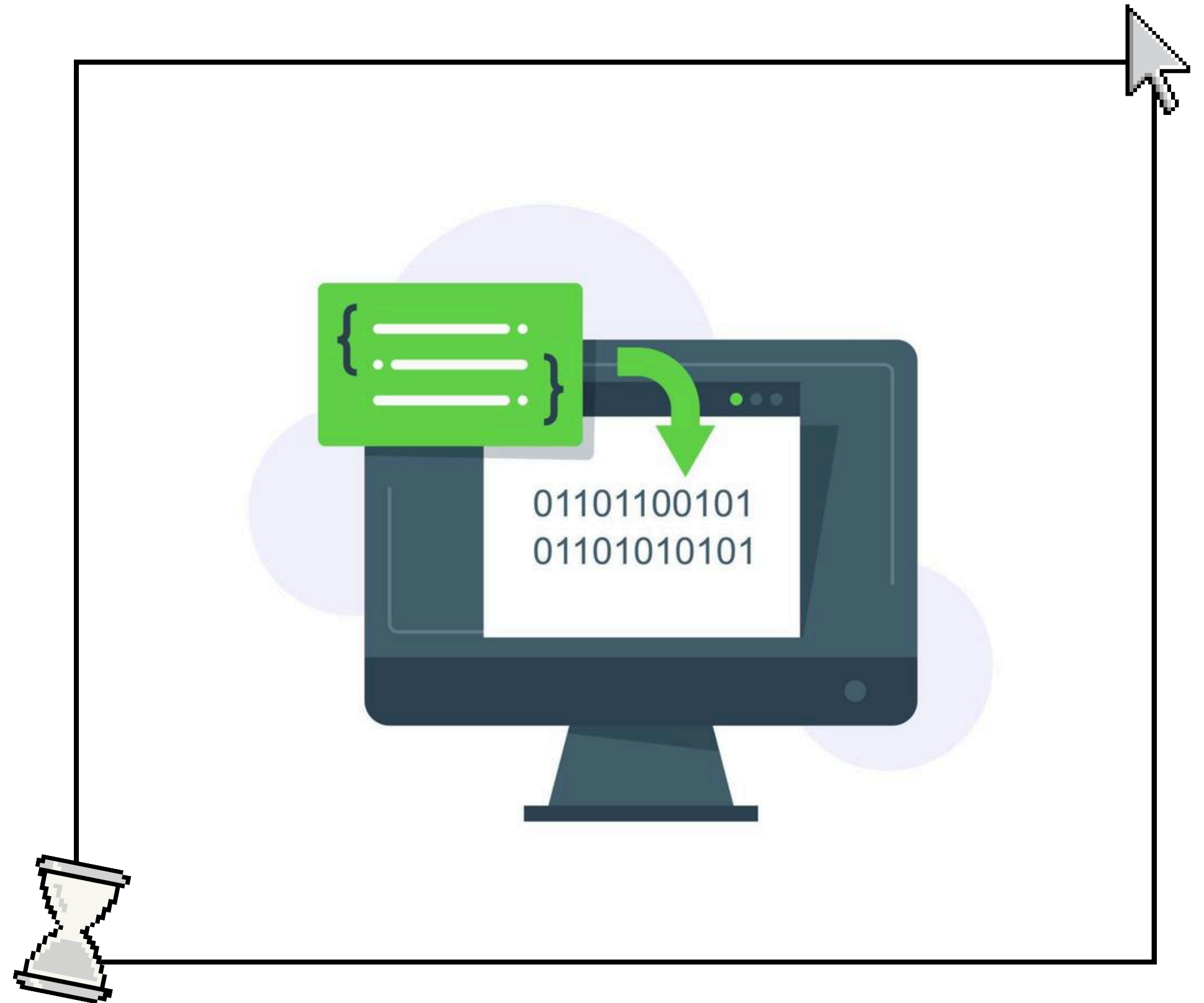
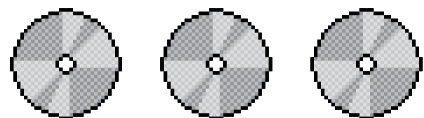
MODELOS IA EN COMPILADORES

El uso de modelos IA para la optimización de compiladores.

Astorga Ramos Jorge Abel

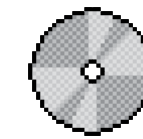
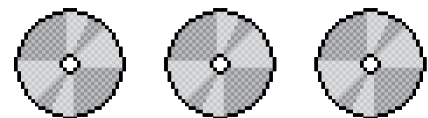
INTRODUCCIÓN

- Los compiladores son programas que traducen código fuente a código ejecutable.
- Tradicionalmente, los compiladores utilizan reglas fijas y heurísticas creadas manualmente por expertos.
- Estas heurísticas ayudan a decidir qué optimizaciones aplicar al código.



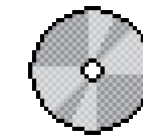
PROBLEMÁTICA ACTUAL

Un compilador puede aplicar muchas optimizaciones diferentes sobre el mismo código.



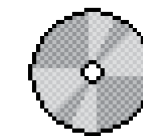
1.-

Las arquitecturas actuales son mucho más complejas, por lo que resulta difícil encontrar la mejor combinación de optimizaciones.



2.-

La mejor opción depende del procesador, la memoria, el tamaño de los datos y el tipo de aplicación.



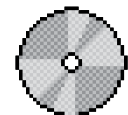
3.-

Elegir manualmente la mejor secuencia de optimizaciones puede consumir mucho tiempo y recursos.

TRANSFORMACIONES DE CÓDIGO

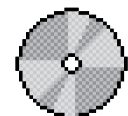
Las transformaciones son cambios aplicados al código para mejorar su rendimiento.

Algunas de las más importantes son:



Paralelización

Divide tareas para ejecutarlas al mismo tiempo.



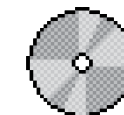
Tiling

Divide grandes bloques de datos en partes más pequeñas.



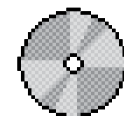
Unrolling

Reduce la cantidad de iteraciones de un ciclo.



Intercambio de bucles

Cambia el orden de los bucles para mejorar acceso a memoria.



Vectorización

Permite ejecutar varias operaciones con una sola instrucción.

PAPEL DE LA IA EN LOS COMPILADORES

Optimizaciones

La IA ayuda a predecir qué optimizaciones generarán mejor rendimiento.

Deep Learning

Los modelos pueden aprender a partir de miles de ejemplos de código optimizado.

Eficiencia

En lugar de probar todas las combinaciones posibles, el sistema puede seleccionar rápidamente las mejores.

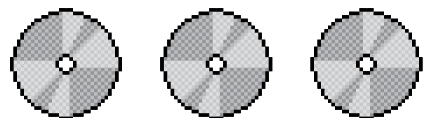
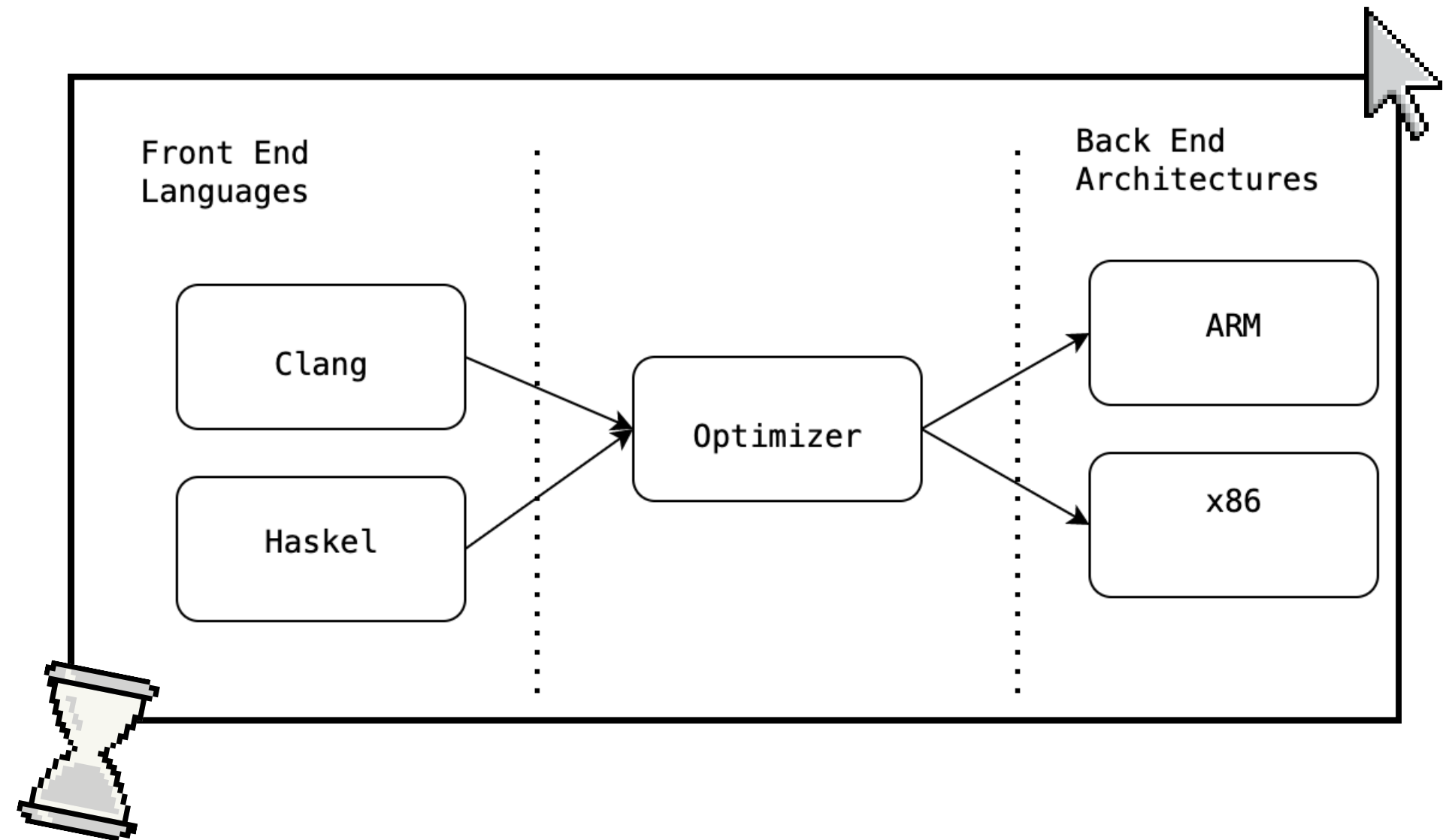
Esto reduce el tiempo de compilación y mejora la eficiencia del programa final. Además, la IA permite adaptar las optimizaciones a diferentes plataformas como CPU, GPU y FPGA.

COMPILADORES

LLVM

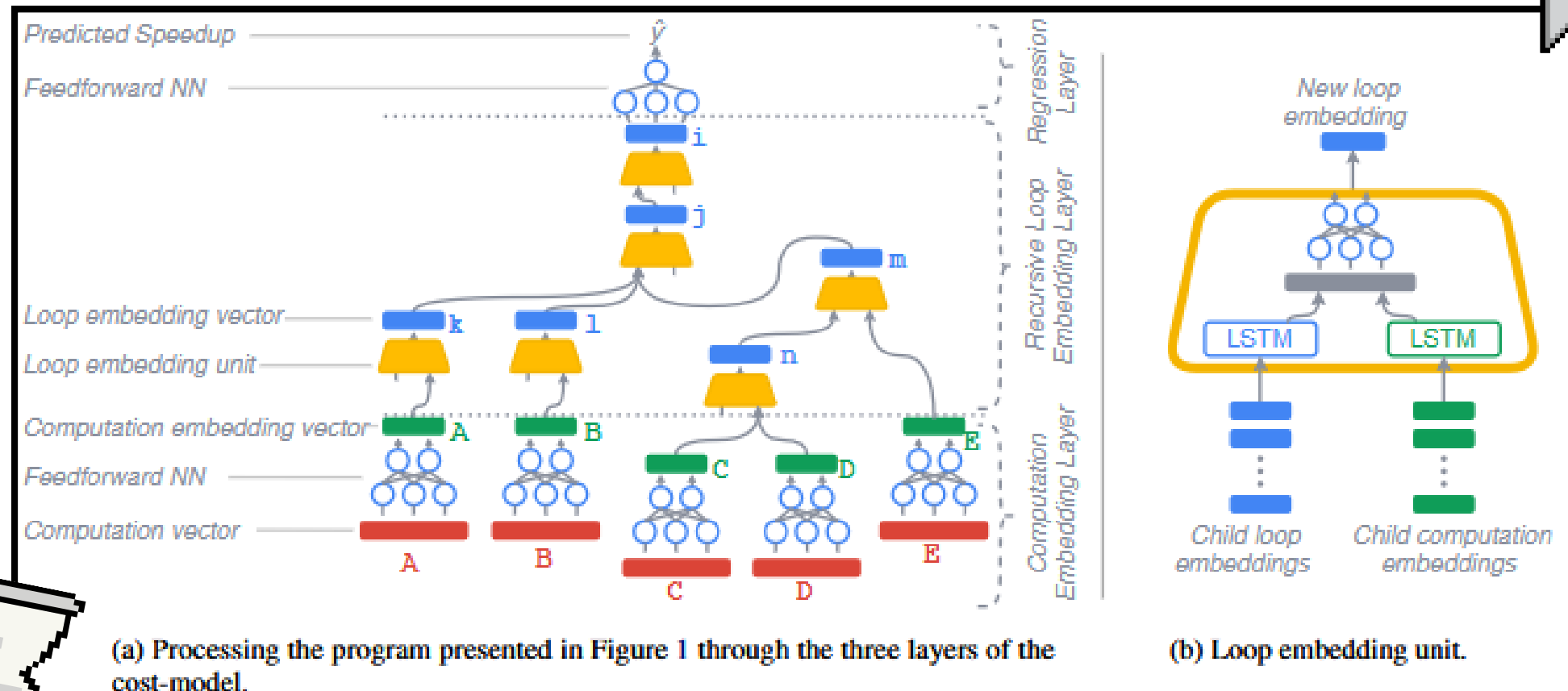
LLVM es una infraestructura de compiladores modular y de código abierto diseñada para facilitar la optimización de programas en tiempo de compilación, enlace y ejecución.

- LLVM MLGO
- Clang
- Google ML Compiler Opt
- Meta LLM
- ACPO



MODELO BASADO EN DEEP LEARNING

Uno de los artículos propone un modelo de Deep Learning aplicado al compilador TIRAMISU. Este compilador trabaja principalmente con programas que contienen múltiples bucles anidados.

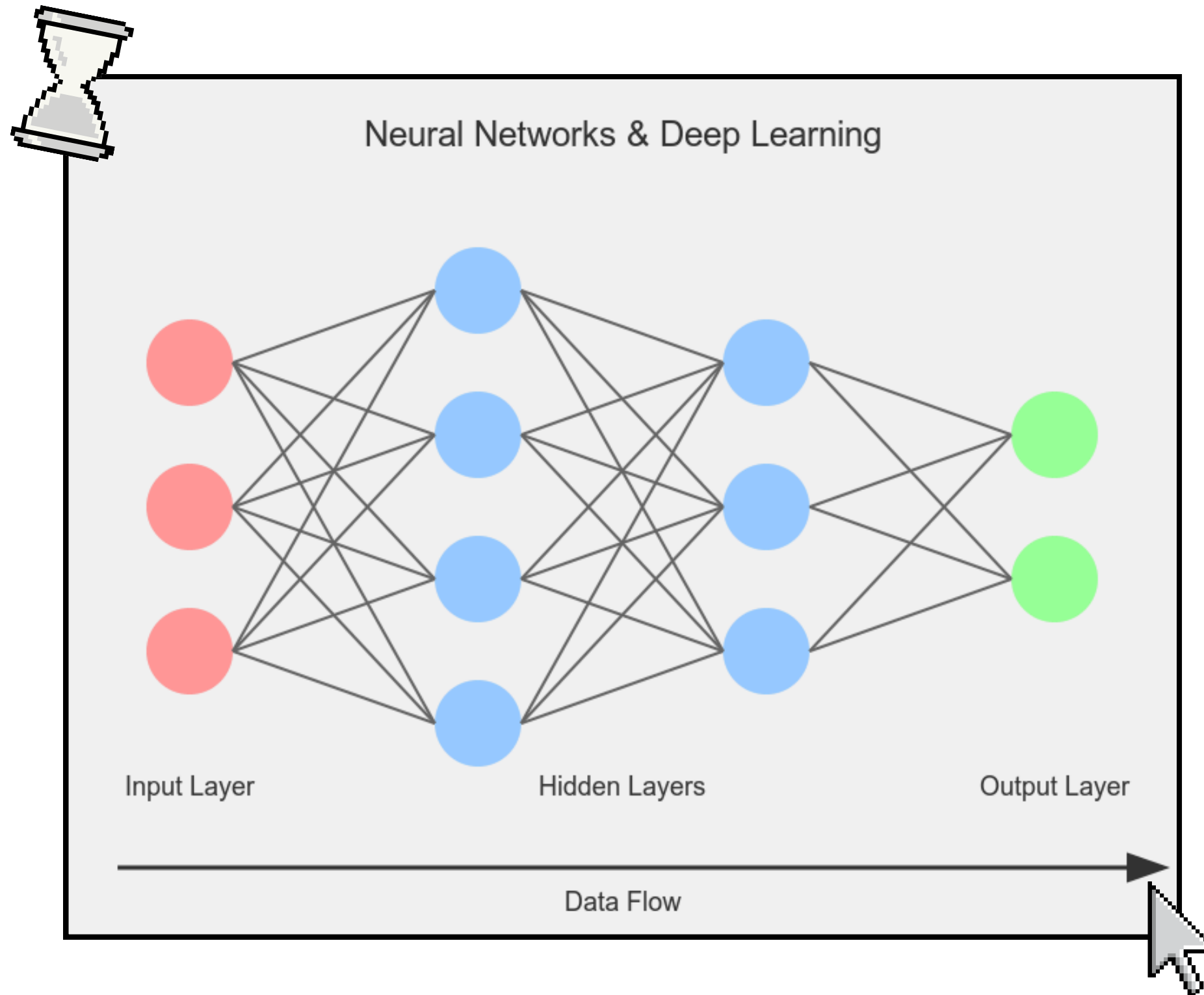


Sistema

El modelo utiliza redes neuronales profundas para estimar el speedup o mejora de rendimiento.

Función

El sistema analiza la estructura del código y predice cuál transformación será más conveniente.



ARQUITECTURA DEL MODELO

El modelo de Deep Learning obtiene información de la estructura interna del programa.

Después se utilizan redes neuronales tipo Feedforward y LSTM. Estas redes aprenden patrones sobre qué transformaciones funcionan mejor en ciertos contextos.

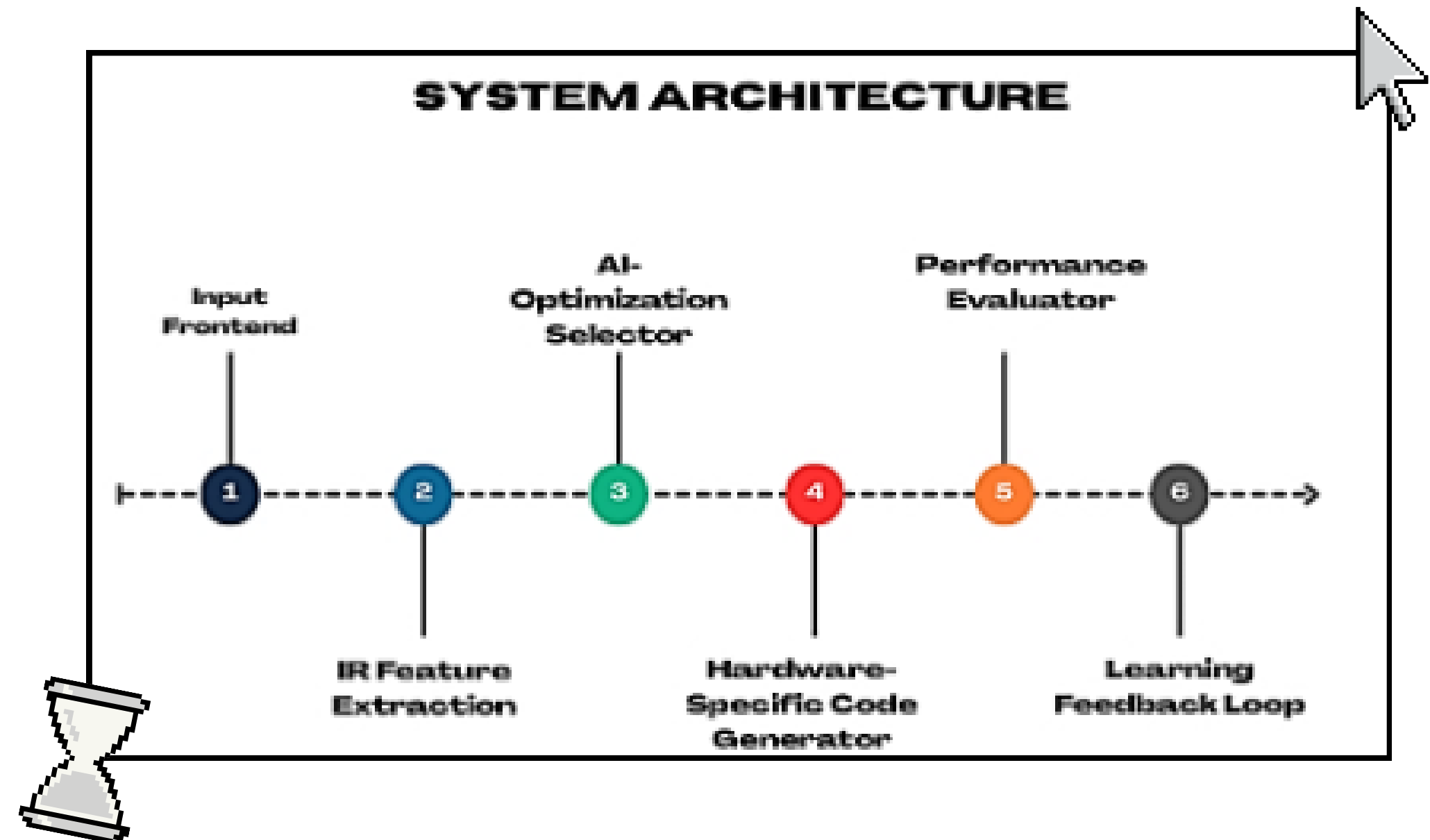
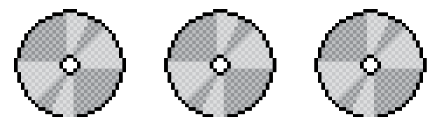
Finalmente, el modelo genera una predicción del rendimiento esperado.

FRAMEWORK

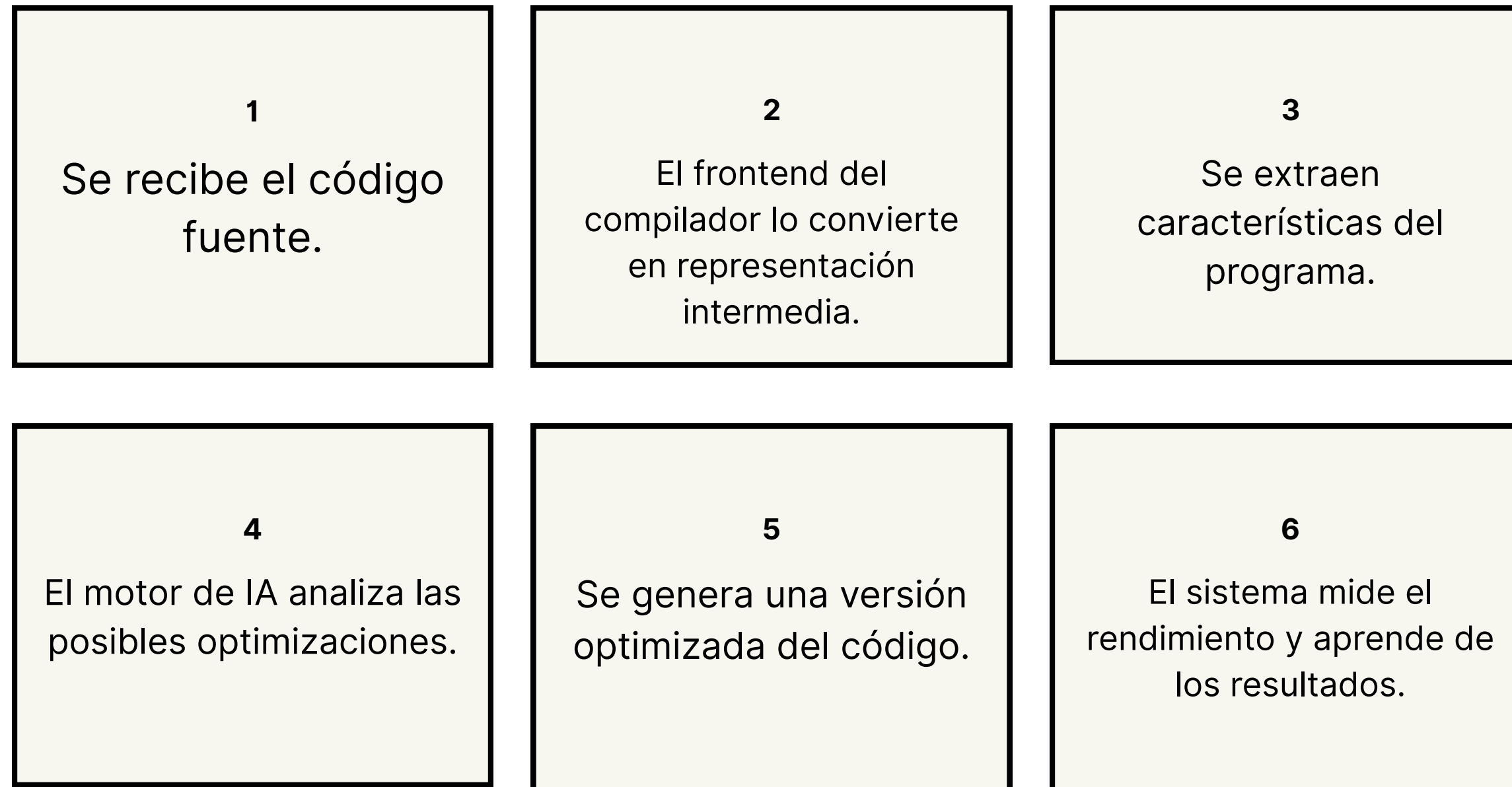
AIOF

Este framework combina Machine Learning con Deep Reinforcement Learning.

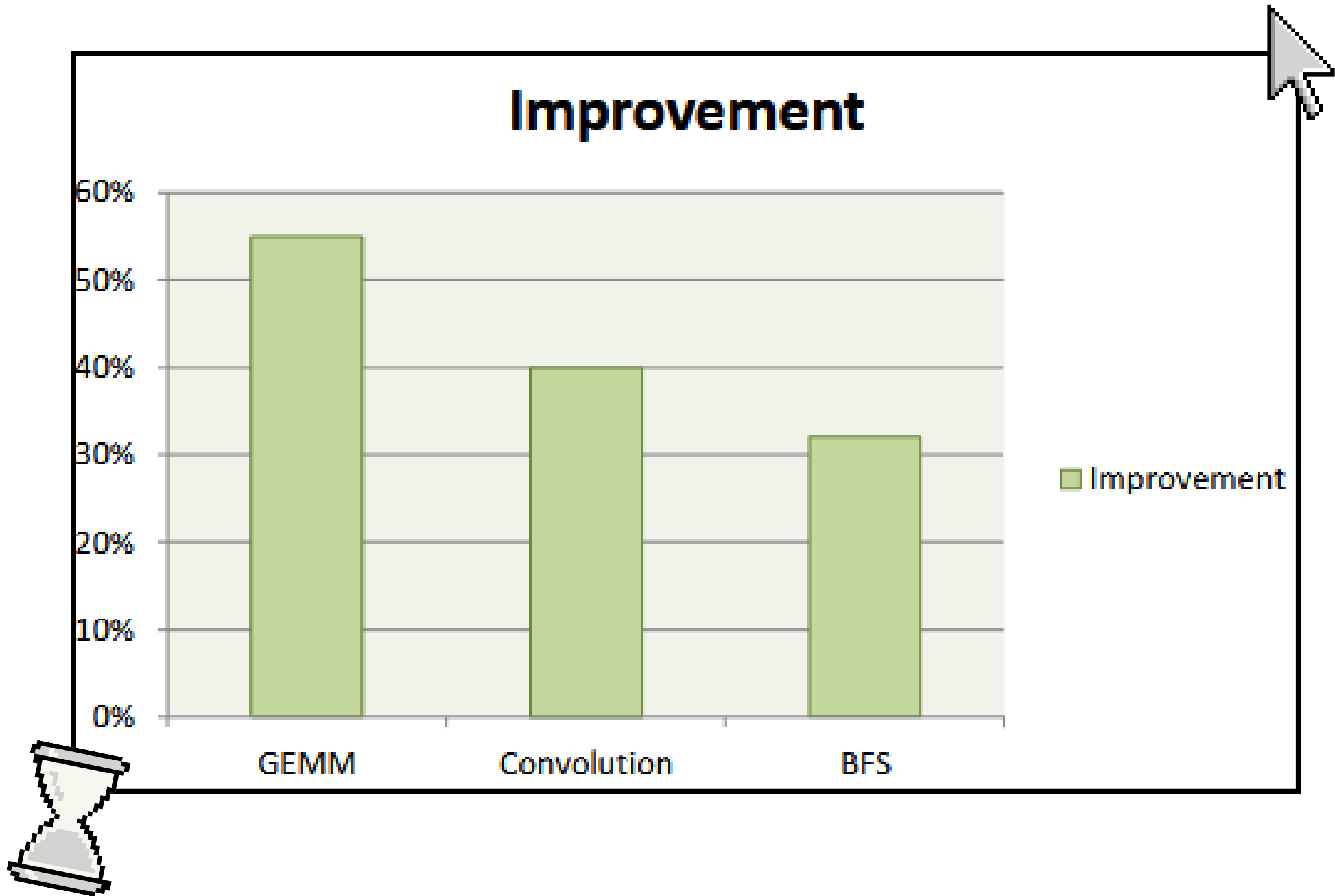
Su objetivo es encontrar automáticamente la mejor secuencia de optimizaciones. Funciona sobre la representación intermedia del compilador, también conocida como IR.



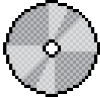
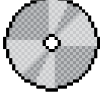
FUNCIONAMIENTO DEL FRAMEWORK



Este proceso se repite para mejorar continuamente las decisiones del compilador.



RESULTADOS OBTENIDOS

-  **GEMM**
Mejora de aproximadamente 55%.
-  **Convolution**
Mejora cercana al 40%.
-  **BFS**
Mejora de aproximadamente 32%.

También se observó una reducción de alrededor del 35% en el tiempo necesario para encontrar optimizaciones. Esto demuestra que la IA puede superar a las heurísticas tradicionales.

HARDWARE EN LOS COMPILADORES

Análisis del hardware

Los compiladores modernos detectan si el sistema tiene CPU, GPU o NPU para decidir cómo optimizar el código.

Tecnologías de aceleración

Funciones como SIMD, AVX, Tensor Cores y Neural Engine permiten ejecutar varias operaciones al mismo tiempo y mejorar el rendimiento.

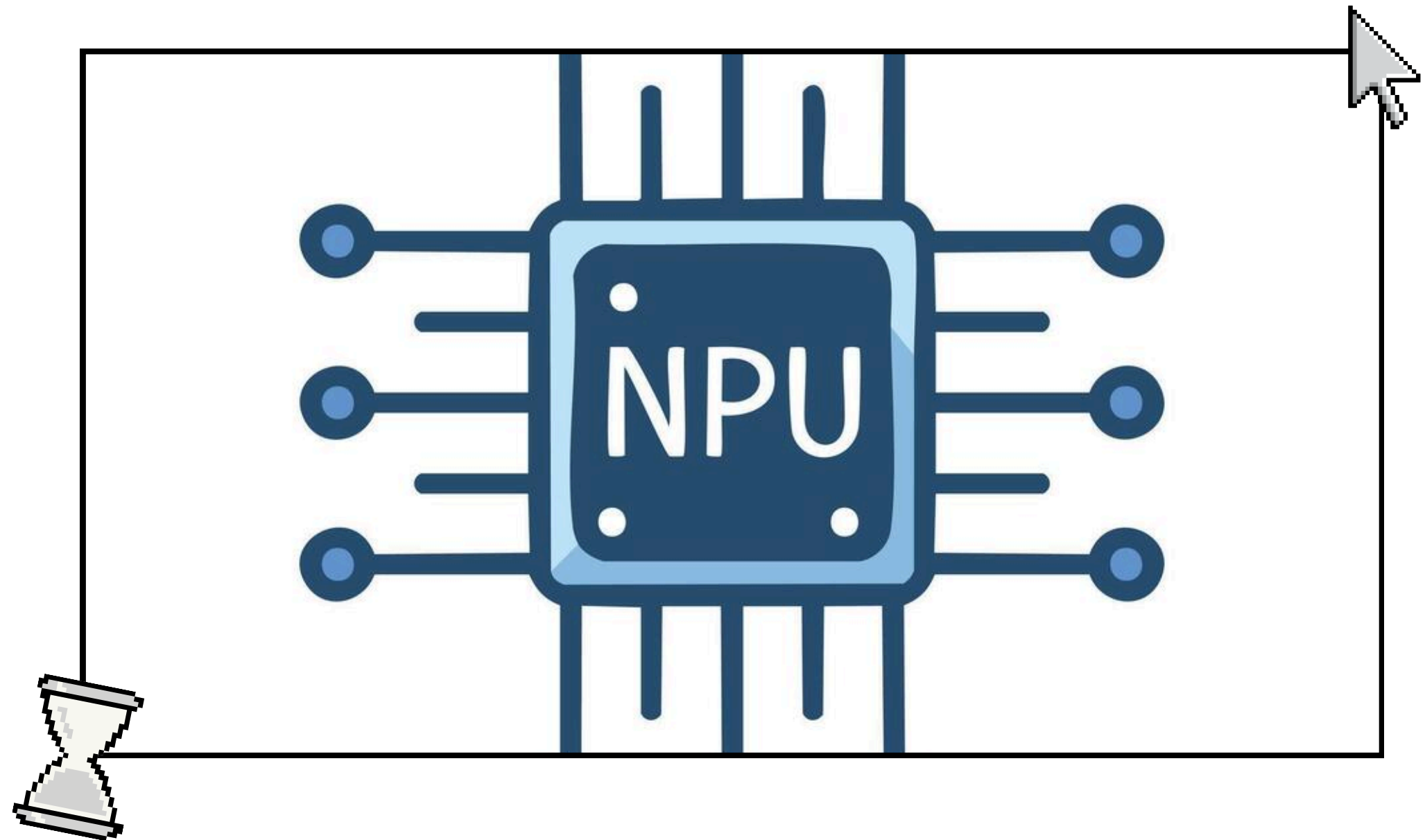
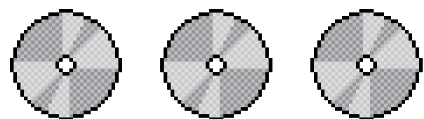
Distribución inteligente de tareas

Dependiendo del hardware disponible, el compilador puede aplicar vectorización, paralelización, unrolling o instrucciones especiales para mejorar la ejecución del programa.

NPU

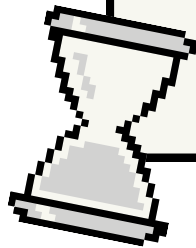
Una NPU es un procesador especializado diseñado para ejecutar operaciones de Inteligencia Artificial, especialmente redes neuronales, de forma más rápida y eficiente que una CPU tradicional.

- Multiplicación de matrices
- Convoluciones
- Operaciones tensoriales
- Vectorización
- Inferencia de modelos



Pseudocodigo

cargar imagen
aplicar convolucion
detectar bordes
mostrar resultado



¿COMO SE RELACIONA?

Un compilador basado en modelos IA puede aprender que:

- Una convolución funciona mejor en la NPU
- Un algoritmo con muchas ramas funciona mejor en CPU
- Una multiplicación de matrices grande funciona mejor en GPU
- Una suma pequeña ni siquiera vale la pena moverla fuera de CPU

EJEMPLO DEEP LEARNING

Codigo - C

```
for(i=0; i<8; i++){  
  for(j=0; j<8; j++){  
    C[i][j] = A[i][j] + B[i][j];  
  }  
}
```

Paso 1: El compilador analiza el código

Detecta que existen:

- 2 ciclos anidados
- Operaciones repetitivas
- Uso de matrices
- Accesos continuos a memoria

EJEMPLO DEEP LEARNING

Paso 2: El modelo de Deep Learning extrae características

- Número de ciclos: 2
- Tamaño de matrices: 4x4
- Tipo de operación: suma
- Cantidad de accesos a memoria: alta
- Dependencias entre instrucciones: bajas

Paso 3: El modelo compara con ejemplos previos

El sistema ya aprendió que, en programas parecidos, la vectorización suele mejorar el rendimiento.

EJEMPLO DEEP LEARNING

Paso 4: El compilador aplica la optimización

Codigo - C

```
#pragma omp parallel for  
for(i=0; i<8; i++){  
    for(j=0; j<8; j++){  
        C[i][j] = A[i][j] + B[i][j];  
    }  
}
```

Paso 5:

En lugar de sumar elemento por elemento, ahora el procesador puede sumar varios valores al mismo tiempo.

EJEMPLO FRAMEWORK

Código - C

```
for(i=0; i<8; i++){  
    for(j=0; j<8; j++){  
        suma += matriz[i][j];  
    }  
}
```

Paso 1: Conversión a IR

El compilador convierte el código a una representación intermedia más fácil de analizar.

```
loop i = 0 to 7  
    loop j = 0 to 7  
        suma = suma + matriz[i][j]
```

EJEMPLO FRAMEWORK

Paso 2: Extracción de características

El framework detecta:

- Dos bucles anidados
- Muchas lecturas de memoria
- Operación repetitiva
- Sin dependencias complejas

Paso 3: Generación de opciones

El framework crea varias posibilidades:

- Opción A: no optimizar
- Opción B: vectorizar
- Opción C: dividir en bloques (tiling)
- Opción D: vectorizar y usar tiling

EJEMPLO FRAMEWORK

Paso 4: Evaluación

El sistema prueba cada opción:

- Opción A: 100 ms
- Opción B: 70 ms
- Opción C: 80 ms
- Opción D: 55 ms

Paso 5: Selección

El framework elige la opción D porque fue la más rápida.

```
for(i=0; i<8; i+=2){  
    for(j=0; j<8; j+=2){  
        procesar_bloque(matriz, i, j);  
    }  
}
```

EJEMPLO REAL

Codigo - C

```
#include <stdio.h>

int main() {
    int A[8] = {1,2,3,4,5,6,7,8};
    int B[8] = {8,7,6,5,4,3,2,1};
    int C[8];

    for(int i = 0; i < 8; i++) {
        C[i] = A[i] + B[i];
    }

    for(int i = 0; i < 8; i++) {
        printf("%d ", C[i]);
    }

    return 0;
}
```

[Ir al sitio WEB](#)

GRACIAS

AI-Driven Optimization of Compiler
Pipelines for Heterogeneous
Processing Architectures

1

<https://doi.org/10.63282/3117-5481/AIJCST-V6I5P102>

Large Language Models for
Compiler Optimization

2 2

<https://doi.org/10.48550/arXiv.2309.07062>

A Deep Learning Based
Cost Model for Automatic
Code Optimization

3 2

<https://doi.org/10.48550/arXiv.2104.04955>